

Module 3: Shell Commands and Regex Parsing

The shell Command

The `shell` command executes commands on the local machine, just like typing them in a terminal:

```
commands:
- type: shell
  cmd: nmap -A -T4 192.168.1.100
```

Options

Option	Default	Description
cmd	(required)	The command to execute
command_shell	/bin/sh	Shell used for execution
interactive	False	Enable interactive mode
creates_session	-	Name for a new interactive session
session	-	Reuse an existing session
command_timeout	15	Seconds to wait in interactive mode

Output Handling

After execution, the command's stdout is stored in `$RESULT_STDOUT` and its return code in `$RESULT_RETURNCODE`. You can also save output directly to a file:

```
commands:
- type: shell
  cmd: nmap -A -T4 172.17.0.106
  save: /tmp/nmap_results.txt
```

Parsing Output with regex

The `regex` command extracts structured data from strings using Python regular expressions. This is essential for chaining attack steps: you scan with `nmap`, parse the output, then use the extracted ports/services in the next step.

New to regular expressions? The [Python regex HOWTO](#) is an excellent introduction. For interactive experimentation, try [regex101.com](#) (select the Python flavor).

Default Behavior: findall

By default, regex uses **findall** mode and reads from **\$RESULT_STDOUT**:

```
commands:
  # Run hydra, output goes to $RESULT_STDOUT
  - type: shell
    cmd: hydra -l user -P ./passwordlist.txt 172.17.0.106 ftp

  # Extract the password from hydra's output
  # Hydra output looks like:
  # "[21][ftp] host: 172.17.0.106 login: user password: user"
  - type: regex
    cmd: ".*login: user.+password: (.+)"
    output:
      USERPW: "$MATCH_0"

  - type: debug
    cmd: "Extracted password: $USERPW"
```

How Matches Work

- Capture groups () in the pattern produce match variables: **\$MATCH_0**, **\$MATCH_1**, etc.
- Without capture groups, the entire match is **\$MATCH_0**
- If the pattern doesn't match, no output variables are set
- **\$REGEX_MATCHES_LIST** contains all matches as a list

The **input** Option

By default, regex operates on **\$RESULT_STDOUT**. Use **input** to read from a different variable (without the **\$**):

```
commands:
  - type: setvar
    variable: MY_DATA
    cmd: "6667/tcp open irc UnrealIRCd"

  - type: regex
    input: MY_DATA
    cmd: (\d+)/tcp
    output:
      PORT: "$MATCH_0"
```

Regex Modes

The **mode** option selects the Python regex function to use:

findall (default): Find all matches

```
- type: regex
  cmd: "666"
  input: MY_DATA
  mode: findall
  output:
    FIRST_MATCH: "$MATCH_0"
```

split: Tokenize a string

Splits the input string at every occurrence of the pattern:

```
# Input: "6667/tcp open irc UnrealIRCd"
# Split on whitespace
- type: regex
  cmd: "\ +"
  input: MY_DATA
  mode: split
  output:
    # MATCH_0 = "6667/tcp"
    # MATCH_1 = "open"
    # MATCH_2 = "irc"
    # MATCH_3 = "UnrealIRCd"
    TOKEN_0: "$MATCH_0"
    TOKEN_2: "$MATCH_2"
```

search: Find first occurrence

Returns the first match anywhere in the string:

```
- type: regex
  cmd: tcp
  input: MY_DATA
  mode: search
  output:
    FOUND: "$MATCH_0"
```

sub: Substitute/replace

Replaces matches with a replacement string:

```
- type: regex
  cmd: tcp
  replace: UDP
  input: MY_DATA
  mode: sub
```

```
output:
# "6667/UDP open irc UnrealIRCd"
MODIFIED: "$MATCH_0"
```

Practical Example: Chaining Shell and Regex

A common pattern is: run a tool, parse its output, use the result in the next step.

```
vars:
  TARGET: 192.168.1.100

commands:
  # Step 1: Scan for open ports
  - type: shell
    cmd: nmap -p 80 $TARGET

  # Step 2: Extract the port number
  - type: regex
    cmd: (\d+)/tcp open http
    output:
      PORT: "$MATCH_0"

  # Step 3: Print it
  - type: debug
    cmd: "Found HTTP on port $PORT"

  # Step 4: Use it in the next command
  - type: shell
    cmd: nikto -host $TARGET -port $PORT
```

The `mktemp` Command

Creates a temporary file or directory that is automatically deleted when AttackMate exits. The path is stored in a variable:

```
commands:
  # Create a temp file
  - type: mktemp
    variable: TEMPFILE

  - type: debug
    cmd: "Temp file created at: $TEMPFILE"

  # Create a temp directory
  - type: mktemp
    cmd: dir
    variable: TEMPDIR
```

This is useful when you need to download or create files as part of an attack (e.g., downloading, generating payloads).